

Level 12: Offline-First Architecture & Local Databases

React Native Masterclass

Expo SDK 54 / RN 0.81

Production Architecture

12.1 — SQLite on Mobile & expo-sqlite

AsyncStorage vs SQLite: AsyncStorage stores data as a flat unindexed JSON string. To find 1 row out of 50,000, it must load all 50,000 records into RAM, causing Out-Of-Memory crashes. SQLite stores data in indexed B-Trees on disk, querying individual rows in 0.5ms with 0% memory waste.

```
import * as SQLite from 'expo-sqlite';

// Open database via C++ JSI bindings
const db = await SQLite.openDatabaseAsync('offline_store.db');

// Parameterized Insert
await db.runAsync('INSERT INTO todos (text, is_synced) VALUES (?, ?);', ['Buy Groceries', 0]);

// Query unsynced rows
const rows = await db.getAllAsync('SELECT * FROM todos WHERE is_synced = 0;');
```

Tenglish Explanation:

AsyncStorage pedda JSON file laga untundi. 50k messages unte okka message vethakadaniki full data RAM lo parse avvali (freeze). SQLite phone disk lo unna real database; `SELECT \* FROM notes WHERE id = 1` query chesthe direct ga 0.5ms lo fetch chesthundi.

12.2 — WatermelonDB & Observable Lazy-Loading

WatermelonDB scales to 100,000+ records at 60 FPS through three core mechanisms:

- **Multithreading via JSI:** SQLite runs on a separate background thread without blocking the JavaScript event loop.
- **Lazy Loading:** Instantiates JavaScript model objects only for items currently visible on the mobile viewport.
- **RxJS Observables:** Components subscribe directly to database rows; only modified records trigger re-renders.

Tenglish Explanation:

WatermelonDB 100,000 items unna sare screen meeda unde 10 items ni mathrame RAM lo load chesthundi (Lazy Loading). SQLite C++ background thread lo run avvadam valla UI ఎప్పటికీ 60 FPS smooth ga untundi.

